

Module-6 Python – Fundamentals of Python Language

1. INTRODUCTION TO PYTHON

THEORY :

1. Introduction to Python and its Features (simple, high-level, interpreted language).



Python is a simple, high-level, interpreted programming language used to develop software, websites, data analysis programs, artificial intelligence applications, and more.

It was created by Guido van Rossum and first released in 1991. Python is popular because it has easy syntax, making it beginner-friendly and widely used in education and industry.

Features of Python

1. Simple and Easy to Learn
Python has a clear and readable syntax similar to English, so beginners can learn it quickly.
2. High-Level Language
Python is a high-level language, which means programmers do not need to manage complex hardware details like memory directly.
3. Interpreted Language
Python code is executed line by line by an interpreter, which makes debugging easier.
4. Object-Oriented
Python supports object-oriented programming (OOP) concepts like classes, objects, inheritance, and polymorphism.
5. Portable
Python programs can run on different operating systems like Windows, Linux, and macOS without major changes.

6. Large Standard Library

Python provides many built-in modules and libraries that help perform tasks such as file handling, web development, and data processing.

7. Open Source

Python is free to use and modify because it is open-source software.

8. Supports Multiple Programming Styles

Python supports procedural, object-oriented, and functional programming.

2. History and evolution of Python.



Python is a popular programming language created by **Guido van Rossum**. It was developed at **Centrum Wiskunde & Informatica (CWI)** in the **Netherlands**. Python was first released in **1991**.

Early Development

- In the late **1980s**, Guido van Rossum started working on Python as a successor to the **ABC Programming Language**.
- His goal was to create a language that was **easy to read, simple to use, and powerful**.

Python 1.x (1991)

- The first version of Python was released in **1991**.
- It included features like **functions, exception handling, and core data types (lists, strings, dictionaries)**.

Python 2.x (2000)

- Python 2.0 was released in **2000**.
- It introduced new features such as:
 - **List comprehensions**
 - **Garbage collection**
 - Better support for Unicode.

Python 3.x (2008 – Present)

- Python 3.0 was released in **2008** as a major improvement over Python 2.
- It fixed many design issues and improved **performance, Unicode support, and consistency**.
- Python 2 officially reached its end of life in **2020**, and now **Python 3** is the standard version used worldwide.

Current Evolution

- Python continues to evolve with new versions that improve **speed, libraries, and functionality**.
- Today it is widely used in fields like **web development, data science, artificial intelligence, and automation**.

3. Advantages of using Python over other programming languages.



1. Simple and Easy to Learn

Python has a very clear and readable syntax similar to English, which makes it easier for beginners to learn compared to many other languages.

2. High-Level Language

Python is a high-level language, so programmers do not need to handle complex tasks like memory management or hardware interaction.

3. Interpreted Language

Python programs run line by line using an interpreter, which makes debugging and testing easier.

4. Less Code Required

Python allows developers to write programs with **fewer lines of code** compared to languages like C or Java, saving time and effort.

5. Extensive Libraries and Frameworks

Python offers a wide range of pre-built libraries and frameworks for various tasks, including:

- Web development: Django, Flask
- Data Science & Machine Learning: Pandas, NumPy, TensorFlow, Scikit-learn
- GUI development: Tkinter, PyQt
- Automation: Selenium, OpenPyXL

6. **Platform Independent**

Python programs can run on different operating systems like Windows, Linux, and macOS without major changes.

7. **Supports Multiple Programming Styles**

Python supports **procedural, object-oriented, and functional programming**, making it flexible for many types of applications.

8. **Large Community Support**

Python has a large global community of developers who create libraries, tutorials, and tools to help others learn and solve problems.

9. **Used in Many Fields**

Python is widely used in **web development, data science, artificial intelligence, automation, and machine learning**.

4. **Installing Python and setting up the development environment (Anaconda, PyCharm, or VS Code).**



Installing Python and Setting Up the Development Environment

To start programming in Python, you must install Python and choose a development environment (IDE or code editor) such as **Anaconda, PyCharm, or Visual Studio Code**.

1. Install Python

1. Go to the official website: <https://www.python.org>
2. Download the latest version of Python for your operating system.
3. Run the installer.
4. Select **"Add Python to PATH"**.
5. Click **Install Now**.

After installation, verify it by opening **Command Prompt** and typing:

```
python --version
```

If the version appears, Python is installed successfully.

Setting Up Development Environments

2. Using Anaconda

Anaconda is mainly used for **data science and machine learning**.

Steps:

1. Download Anaconda from <https://www.anaconda.com>
2. Install it like normal software.
3. Open **Anaconda Navigator**.
4. Launch **Jupyter Notebook** or **Spyder IDE** to start writing Python code.

Advantages:

- Includes many pre-installed libraries (NumPy, Pandas, Matplotlib).
- Good for data analysis and scientific computing.

3. Using PyCharm

PyCharm is a powerful IDE made specifically for Python.

Steps:

1. Download PyCharm from <https://www.jetbrains.com/pycharm>
2. Install and open it.
3. Click **New Project**.
4. Select the installed Python interpreter.
5. Create a new .py file and start coding.

Advantages:

- Intelligent code completion
 - Built-in debugger
 - Professional development tools
-

4. Using Visual Studio Code (VS Code)

VS Code is a lightweight and popular code editor.

Steps:

1. Download from <https://code.visualstudio.com>
2. Install and open it.
3. Install the **Python extension** from Microsoft.
4. Create a .py file and run your Python program.

Advantages:

- Fast and lightweight
- Supports many programming languages
- Large extension marketplace

5. Writing and executing your first Python program.



1. Writing the First Python Program

```
print("Hello, World!")
```

Explanation

- `print()` is a built-in Python function.
- It is used to **display output on the screen**.

2. Saving the Program

Save the file as:

hello.py

3. Executing (Running) the Program

Method : Using Command Prompt / Terminal

1. Open **Command Prompt** or Terminal.
2. Go to the folder where the file is saved.
3. Run the program using:

```
python hello.py
```

Output

Hello, World!

2. PROGRAMMING STYLE

THEORY :

1. Understanding Python's PEP 8 guidelines.



PEP 8 is the **official style guide for writing code in Python**.

It provides rules and recommendations on how to format Python code so that it is **clean, readable, and consistent**.

PEP stands for **Python Enhancement Proposal**.

1. Indentation

- Use **4 spaces** for indentation.
- Do not use tabs.

Example:

```
if x > 5:
```

```
    print("x is greater than 5")
```

2. Maximum Line Length

- Each line should be **maximum 79 characters** long.
- This improves readability on different screens.

3. Naming Conventions

Variables and Functions

- Use **lowercase letters with underscores**.

Example:

```
total_marks = 90
```

```
def calculate_sum():
```


Class Names

- Use **CapitalizedWords (CamelCase)**.

Example:

```
class StudentDetails:
```

4. Spaces in Expressions

Use spaces around operators.

Correct:

```
x = a + b
```

Incorrect:

```
x=a+b
```

5. Comments

- Write comments to explain code clearly.

Example:

```
# This program prints a greeting
```

```
print("Hello")
```

7. Imports

- Imports should usually be written **on separate lines** and at the **top of the file**.

Example:

```
import math
```

```
import os
```

2. Indentation, comments, and naming conventions in Python.



1. Indentation

- Use **4 spaces** for indentation.
- Do not use tabs.

Example:

```
if x > 5:  
    print("x is greater than 5")
```

2. Comments

- Write comments to explain code clearly.

Example:

```
# This program prints a greeting  
print("Hello")
```

3. Naming Conventions

Variables and Functions

- Use **lowercase letters with underscores**.

Example:

```
total_marks = 90  
def calculate_sum():
```

Class Names

- Use **CapitalizedWords (CamelCase)**.

Example:

```
class StudentDetails:
```

3. Indentation, comments, and naming conventions in Python.



1. Use Meaningful Variable Names

Choose names that clearly describe the purpose of the variable.

Example:

Bad:

```
x = 50
```

Good:

```
total_marks = 50
```

2. Follow Proper Indentation

Use **4 spaces for indentation** to clearly show code blocks.

Example:

```
age = 18
```

```
if age >= 18:
```

```
    print("Eligible to vote")
```

3. Write Comments to Explain Code

Comments help others understand what the code is doing.

Example:

```
# Calculate total price
```

```
total_price = quantity * price
```

4. Keep Code Simple

Avoid writing overly complex statements. Break code into small and clear parts.

Example:

```
a = 10
```

```
b = 20
```

```
sum = a + b
```

```
print(sum)
```

5. Use Functions

Divide large programs into smaller functions for better organization.

Example:

```
def greet():
```

```
    print("Hello")
```

```
greet()
```

6. Maintain Consistent Formatting

Keep consistent spacing, naming style, and structure throughout the program.

3. CORE PYTHON CONCEPTS

THEORY :

1. Understanding data types: integers, floats, strings, lists, tuples, dictionaries, sets.



Data types define the type of value a variable can store in a program.

1. Integer (int)

An **integer** is a whole number without a decimal point.

Example:

```
age = 20
```

```
marks = 95
```

Examples of integers: **-5, 0, 10, 100**

2. Float (float)

A **float** is a number with a **decimal point**.

Example:

```
price = 99.50
```

```
temperature = 36.6
```

3. String (str)

A **string** is a sequence of characters enclosed in **single or double quotes**.

Example:

```
name = "Apple"
```

```
message = 'Hello'
```

Strings are used to store **text data**.

4. List

A **list** is a collection of items stored in **square brackets []**.

Lists are **ordered and changeable (mutable)**.

Example:

```
numbers = [1, 2, 3, 4]
```

```
names = ["Ali", "John", "Sara"]
```

5. Tuple

A **tuple** is similar to a list but stored in **parentheses ()**.

Tuples are **ordered but cannot be changed (immutable)**.

Example:

```
coordinates = (10, 20)
```

```
colors = ("red", "green", "blue")
```

6. Dictionary

A **dictionary** stores data in **key–value pairs** using **curly braces { }**.

Example:

```
student = {  
    "name": "XYZ",  
    "age": 20,  
    "marks": 90  
}
```

7. Set

A **set** is a collection of **unique elements** stored in **curly braces { }**.

Example:

```
numbers = {1, 2, 3, 4}
```

Features:

- No duplicate values
- Unordered collection

2. Python variables and memory allocation.



1. Python Variables

A **variable** is a name used to store a value in memory.

Example:

```
x = 10
```

```
name = "Hello"
```

```
price = 99.5
```

Here:

- x stores an integer
- name stores a string
- price stores a float

2. Rules for Naming Variables

- Variable Names Must Begin with a Letter or Underscore
- Cannot start with a number
- Variable Names Can Contain Letters, Numbers, and Underscores
- Variable Names Are Case-Sensitive
- Variable Names Cannot Be Reserved Keywords
- Variable Names Should Be Meaningful

Example:

```
age = 20
```

```
student_name = "John"
```

3. Memory Allocation in Python

Memory allocation means reserving space in the computer's memory to store values.

In Python:

- Memory is **automatically managed** by Python.
- When a variable is created, Python **allocates memory for the value**.
- Python uses **dynamic typing**, meaning the variable type is decided at runtime.

Example:

```
a = 10
```

```
b = "Hello"
```

Here:

- Memory is allocated to store the integer 10.
- Memory is allocated to store the string "Hello".

4. Dynamic Memory Allocation

Python allows variables to **change their type during execution**.

Example:

```
x = 10
```

```
x = "Hello"
```

First x stores an **integer**, then it stores a **string**. Python automatically manages the memory for these values.

3. Python operators: arithmetic, comparison, logical, bitwise.



Operators are symbols used to **perform operations on variables and values**.

1. Arithmetic Operators

Used to perform **mathematical calculations**.

Operator	Meaning	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus (remainder)	a % b
**	Exponent (power)	a ** b
//	Floor division	a // b

Example:

```
a = 10
```

```
b = 3
```

```
print(a + b)
```

```
print(a % b)
```

2. Comparison Operators

Used to **compare two values** and return **True or False**.

Operator	Meaning
==	Equal to
!=	Not equal to

Operator	Meaning
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example:

```
a = 10
```

```
b = 5
```

```
print(a > b)
```

```
print(a == b)
```

3. Logical Operators

Used to combine **multiple conditions**.

Operator	Meaning
and	True if both conditions are true
or	True if at least one condition is true
not	Reverses the result

Example:

```
a = 10
```

```
b = 5
```

```
print(a > 5 and b < 10)
```

4. Bitwise Operators

Used to perform operations on **binary numbers (bits)**.

Operator	Meaning
&	AND
	OR
^	XOR
~	NOT
<<	Left shift
>>	Right shift

Example:

```
a = 5
```

```
b = 3
```

```
print(a & b)
```

```
print(a | b)
```

4. CONDITIONAL STATEMENTS

THEORY :

1. Introduction to conditional statements: if, else, elif.



Conditional statements are used to **make decisions in a program**. They allow the program to **execute different blocks of code based on conditions**.

1. if Statement

The **if statement** executes a block of code **only if the condition is true**.

Syntax:

```
if condition:  
    statement
```

Example:

```
age = 18  
if age >= 18:  
    print("You are eligible to vote")
```

2. else Statement

The **else statement** executes when the **if condition is false**.

Syntax:

```
if condition:  
    statement1  
else:  
    statement2
```

Example:

```
age = 16
if age >= 18:
    print("Eligible to vote")
else:
    print("Not eligible to vote")
```

3. elif Statement

The **elif (else if)** statement checks **multiple conditions**.

Syntax:

```
if condition1:
    statement1
elif condition2:
    statement2
else:
    statement3
```

Example:

```
marks = 75

if marks >= 90:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

2. Nested if-else conditions.



A **nested if-else** means using an **if or else statement inside another if or else block**.

It is used when a program needs to **check multiple conditions step by step**.

Syntax

```
if condition1:
    if condition2:
        statement1
    else:
        statement2
else:
    statement3
```

Example Program

```
age = 20
citizen = True

if age >= 18:
    if citizen:
        print("Eligible to vote")
    else:
        print("Not a citizen")
else:
    print("Under 18")
```

Output

Eligible to vote

5. LOOPING (FOR , WHILE)

THEORY :

1. Introduction to for and while loops.



Loops are used to **repeat a block of code multiple times** until a condition is satisfied.

1. for Loop

A **for loop** is used to **iterate over a sequence** such as a list, tuple, string, or range of numbers.

Syntax

for variable in sequence:

 statement

Example

```
for i in range(5):
```

```
    print(i)
```

Output

0

1

2

3

4

Explanation:

- range(5) generates numbers from **0 to 4**.
- The loop prints each number one by one.

2. while Loop

A **while loop** executes a block of code **as long as the condition is true**.

Syntax

```
while condition:  
    statement
```

Example

```
i = 1  
while i <= 5:  
    print(i)  
    i += 1
```

Output

```
1  
2  
3  
4  
5
```

Explanation:

- The loop runs while $i \leq 5$.
- After each iteration, i increases by 1.

Summary:

- **for loop** → used when the number of iterations is known.
- **while loop** → used when the loop runs until a condition becomes false.

2. How loops work in Python.



Loops are used to **repeat a block of code multiple times** until a condition is satisfied.

Loops help avoid writing the same code many times.

Python mainly uses **for loops** and **while loops**.

Basic Working of a Loop

A loop generally works in these steps:

1. **Initialization** – A variable is given a starting value.
2. **Condition Check** – The loop checks whether the condition is true or false.
3. **Execution** – If the condition is true, the code inside the loop runs.
4. **Update** – The loop variable is updated (increased or decreased).
5. **Repeat or Stop** – The loop repeats until the condition becomes false.

3. Using loops with collections (lists, tuples, etc.).



In Python, **loops** can be used to **iterate (go through) each element of a collection** such as **lists, tuples, strings, sets, or dictionaries**.

The **for loop** is commonly used for this purpose.

1. Loop with a List

A **list** stores multiple items in square brackets [].

Example:

```
numbers = [1, 2, 3, 4]
```

```
for num in numbers:  
    print(num)
```

Output:

```
1  
2  
3  
4
```

2. Loop with a Tuple

A **tuple** is similar to a list but **cannot be changed**.

Example:

```
colors = ("red", "green", "blue")
```

```
for color in colors:  
    print(color)
```

Output:

```
red  
green  
blue
```

3. Loop with a String

A **string** is a sequence of characters.

Example:

```
name = "Python"
```

```
for letter in name:
```

```
    print(letter)
```

Output:

P

y

t

h

o

n

4. Loop with a Dictionary

A **dictionary** stores data in **key–value pairs**.

Example:

```
student = {"name": "John", "age": 20}
```

```
for key in student:
```

```
    print(key, student[key])
```

Output:

name John

age 20

8. CONTROL STATEMENTS (BREAK , CONTINUE , PASS)

THEORY :

1. Understanding the role of break, continue, and pass in Python loops.



These statements are used to **control the flow of loops** in Python.

1. break Statement

The **break** statement is used to **exit the loop immediately**, even if the loop condition is still true.

Example

```
for i in range(5):
```

```
    if i == 3:
```

```
        break
```

```
    print(i)
```

Output

0

1

2

Explanation:

When i becomes **3**, the break statement stops the loop completely.

2. continue Statement

The **continue** statement **skips the current iteration** and moves to the **next iteration of the loop**.

Example

```
for i in range(5):
```

```
    if i == 3:
```

```
        continue
```

```
    print(i)
```

Output

0

1

2

4

Explanation:

When **i** is **3**, the program skips printing it and continues with the next value.

3. pass Statement

The **pass** statement is a **null statement**.

It does **nothing**, but it is used as a **placeholder when a statement is required syntactically**.

Example

```
for i in range(5):
```

```
    if i == 3:
```

```
        pass
```

```
    print(i)
```

Output

0

1

2

3

4

Explanation:

When i is 3, pass does nothing and the loop continues normally.

9. STRING MANIPULATION

THEORY :

1. Understanding how to access and manipulate strings.



A **string** is a sequence of characters used to store text.

Strings are written inside **single quotes** (' ') or **double quotes** (" ").

Example:

```
text = "Hello Python"
```

1. Accessing Characters in a String (Indexing)

Each character in a string has an **index number** starting from **0**.

Example:

```
text = "Python"
```

```
print(text[0])
```

```
print(text[3])
```

Output:

P

H

2. Negative Indexing

Python also allows **negative indexing** to access characters from the end.

Example:

```
text = "Python"
```

```
print(text[-1])
```

Output:

N

3. String Slicing

Slicing extracts a part of the string.

Syntax:

```
string[start:end]
```

Example:

```
text = "Python"
```

```
print(text[0:3])
```

Output:

```
Pyt
```

4. String Concatenation

Concatenation means **joining two strings** using +.

Example:

```
first = "Hello"
```

```
second = "World"
```

```
print(first + " " + second)
```

Output:

```
Hello World
```

5. String Repetition

A string can be repeated using *.

Example:

```
print("Hi " * 3)
```

Output:

```
Hi Hi Hi
```


6. String Methods (Manipulation)

Python provides built-in methods to modify strings.

Example:

```
text = "python"  
print(text.upper())  
print(text.lower())  
print(text.replace("python", "Java"))
```

Output:

```
PYTHON  
python  
Java
```

2. Basic operations: concatenation, repetition, string methods (upper(), lower(), etc.).

→

1. String Concatenation

Concatenation means **joining two strings** using +.

Example:

```
first = "Hello"  
second = "World"  
print(first + " " + second)
```

Output:

```
Hello World
```

2. String Repetition

A string can be repeated using `*`.

Example:

```
print("Hi " * 3)
```

Output:

Hi Hi Hi

3. String Methods

Python provides **built-in methods** to manipulate strings.

Example:

```
text = "python"  
print(text.upper())  
print(text.lower())
```

Output:

PYTHON
python

3. String slicing.

→ **Slicing** extracts a part of the string.

Syntax:

```
string[start:end]
```

Example:

```
text = "Python"  
print(text[0:3])
```

Output:

Pyt